Express.js — Day 12

Error Handling • Long-Term Reference Notes

Day 12 Goal

Understand Express error handling, **next(error)**, error-handling middleware, **err.message**, and why the error handler is normally registered after routes.

1. What is an Express error?

An error can occur while processing a request. The goal of error handling is to turn failures into controlled responses instead of leaving requests unhandled.

```
app.get("/user", (req, res) => {
const user = null;
res.json(user.name);
});
```

2. Error-handling middleware

Normal middleware uses **(req, res, next)**. Error-handling middleware uses four parameters: **(err, req, res, next)**. The **err** parameter identifies the function as error-handling middleware.

```
const errorHandler = (err, req, res, next) => {
res.status(500).send("Something went wrong");
};
```

3. next() vs next(error)

```
next();
```

next() continues the normal middleware chain.

```
next(error);
```

next(error) signals that an error occurred and moves Express into its error-handling flow.

4. Creating and passing an error

```
app.get("/test", (req, res, next) => {
const error = new Error("Something went wrong");
next(error);
});
```

The Error object contains the message. **next(error)** passes that same Error object forward.

5. Registering the error handler

```
const errorHandler = (err, req, res, next) => {
console.log(err.message);
res.status(500).send("Something went wrong");
};

app.use(errorHandler);
```

next(error) does not magically call a function named **errorHandler**. It puts Express into error-handling mode, and Express looks for matching error-handling middleware in the chain.

6. Complete request flow

```
GET /test
↓
Route handler
↓
new Error(...)
↓
next(error)
↓
Error-handling middleware
↓
500 response
```

7. Why err.message works

```
const error = new Error("Something went wrong");
next(error);
```

Because the same Error object is passed to the error handler, **err.message** contains **Something went wrong**. Our terminal output confirmed this.

```
console.log(err.message);
```

In a simple example we could send **err.message** to the client, but real applications often log detailed internal errors and send a safer generic message such as **Internal server error**.

8. Why is the error handler after the routes?

Express processes middleware in registration order. Routes and normal middleware that can generate errors are normally registered first, followed by the final error handler.

```
// routes / normal middleware
app.get("/test", ...);
app.post("/users", ...);

// error handler — normally last
app.use(errorHandler);
```

If a route later calls **next(error)**, Express can continue through the chain and reach the error handler. Putting the error handler at the top would usually place it before the route that generates the error, so that request would have already passed it.

9. Day 12 practical test

You ran the server and tested `GET http://localhost:3000/test` in Postman.

Observed result: **500 Internal Server Error** with **Something went wrong**. The terminal also printed **Something went wrong** from `err.message`.

10. Day 12 assessment — reference answers

1. `next()` continues the normal chain; `next(error)` signals an error and moves into error handling.

2. Error-handling middleware has four parameters: `(err, req, res, next)`.

3. Express recognizes it by the four-parameter error-handler signature beginning with `err`.

11. Long-term mental model

```
Normal flow:
Request → Middleware/Route → next() → Next handler

Error flow:
Request → Middleware/Route → next(error) → Error handler → Error response
```

Key rule: normal middleware uses `next()`; error-producing code uses `next(error)`; the error handler receives the error and decides how to respond.

Express.js Day 12 — COMPLETE ■

You learned and tested Express error handling, including `next(error)`, error-handling middleware, `err.message`, and middleware ordering.